PROJECT CODING GUIDE

Motor Insurance Pricing

End-to-end coding techniques: data analysis to final model implementation

Project: Improving Motor Insurance Pricing Through Joint Modelling of Claim Frequency and Claim Severity

Dataset: freMTPL2 French Motor Third-Party Liability (MTPL)

> This guide is a practical workflow and code reference for your project. It covers reproducible data loading, reconciliation, exploratory analysis, actuarial baselines, machine-learning models, validation, interpretation, and scoring. Run each stage in order and document decisions; do not treat code examples as evidence that a model has already been fitted or validated.

Core stack: Python, pandas, NumPy, scikit-learn, statsmodels, XGBoost, matplotlib/seaborn, SHAP, and joblib. Optional packages should be installed only when their stage is reached.

Contents

1. Modeling workflow and target definitions

2. Project setup and reproducibility
3. Load, validate, and reconcile the files
4. Exploratory data analysis
5. Split data and prevent leakage
6. Actuarial baselines: Poisson frequency and Gamma severity
7. Machine learning and pure-premium modeling
8. Evaluation, calibration, and model selection
9. Interpretability and group analysis
10. Final fit, save, score, and report
11. Practical checklist and research cautions

1. Modeling workflow and target definitions

The project has a policy-level file (`freMTPL2freq.csv`) and a claim-level file (`freMTPL2sev.csv`). Keep these levels distinct until their records have been checked and reconciled.

Quantity	Definition	Unit / use
Claim frequency rate	Claim count divided by exposure	Expected claims per policy-year
Expected claim count	Frequency rate multiplied by exposure	Expected claims during that policy's observed exposure
Average severity	Claim amount averaged over claims	Monetary amount per claim
Pure premium	Frequency rate multiplied by average severity	Expected claim cost per unit of exposure, before expenses, commissions, profit, or taxes

An alternative policy-level target is total claim cost divided by exposure. This supports a Tweedie model, which can represent a mass at zero and positive continuous costs. Confirm the currency/unit of claim amounts before interpreting prices.

> **Research gate:** reconcile claim IDs and counts before creating severity targets or policy total losses. Do not silently drop unmatched claims, replace them with zero, or assume every frequency count has a matching severity record.

2. Project setup and reproducibility

Use paths relative to the script rather than the current terminal directory. Keep raw data unchanged; write derived data and outputs to separate folders. Fix random seeds for repeatable splits and models, while recording package versions and the data source.

```
from pathlib import Path
import platform
import numpy as np
import pandas as pd
```

```
import sklearn

ROOT = Path(__file__).resolve().parent
DATA_DIR = ROOT / "Data"
REPORT_DIR = ROOT / "Reports"
REPORT_DIR.mkdir(exist_ok=True)

SEED = 42
np.random.seed(SEED)
print("Python:", platform.python_version())
print("pandas:", pd.__version__, "scikit-learn:", sklearn.__version__)
```

On Windows PowerShell, create an isolated environment before installing project packages. Install optional libraries only if you reach those modeling stages.

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install pandas numpy matplotlib seaborn scikit-learn statsmodels xgboost joblib
# Optional:
python -m pip install lightgbm shap
# After confirming the environment, record the installed versions:
python -m pip freeze > requirements.txt
```

For notebooks, set `ROOT` explicitly or move reusable logic into a Python module. Avoid machine-specific absolute paths in code that will be shared or rerun.

3. Load, validate, and reconcile the files

Load policy IDs as strings first. Spreadsheet exports can render an ID such as `1.00E+05`; parse it numerically and verify it is an integer before converting. A float conversion followed by an unchecked cast can corrupt identifiers.

```
from decimal import Decimal

freq = pd.read_csv(DATA_DIR / "freMTPL2freq.csv", dtype={"IDpol": "string"})
sev = pd.read_csv(DATA_DIR / "freMTPL2sev.csv", dtype={"IDpol": "string"})

def parse_policy_id(value):
    number = Decimal(str(value).strip())
    if number != number.to_integral_value():
        raise ValueError(f"Non-integer policy ID: {value!r}")
    return int(number)

freq["IDpol"] = freq["IDpol"].map(parse_policy_id).astype("int64")
sev["IDpol"] = sev["IDpol"].map(parse_policy_id).astype("int64")
freq["ClaimNb"] = pd.to_numeric(freq["ClaimNb"], errors="raise").astype("int64")
freq["Exposure"] = pd.to_numeric(freq["Exposure"], errors="raise")
sev["ClaimAmount"] = pd.to_numeric(sev["ClaimAmount"], errors="raise")

assert freq["IDpol"].is_unique
assert freq["Exposure"].gt(0).all()
assert freq["ClaimNb"].ge(0).all()
assert sev["ClaimAmount"].gt(0).all()
print(freq.shape, sev.shape)
print(freq.isna().sum(), sev.isna().sum())
```

Then aggregate claim rows to policy level and compare to the frequency file. Keep a reconciliation table and decide explicitly how each discrepancy is handled, based on the source documentation and research protocol.

```
sev_by_policy = (
    sev.groupby("IDpol", as_index=False)
       .agg(severity_rows=("ClaimAmount", "size"),
            total_loss=("ClaimAmount", "sum"))
)
audit = freq[["IDpol", "ClaimNb"]].merge(
```

```
    sev_by_policy, on="IDpol", how="outer", indicator=True, validate="one_to_one"
)
audit["severity_rows"] = audit["severity_rows"].fillna(0).astype("int64")

print("Frequency-only IDs:", (audit["_merge"] == "left_only").sum())
print("Severity-only IDs:", (audit["_merge"] == "right_only").sum())
matched = audit["_merge"].eq("both")
print("Matched count mismatches:",
      (audit.loc[matched, "ClaimNb"] !=
       audit.loc[matched, "severity_rows"]).sum())
print("Rows with missing values by file:")
print(freq.isna().sum(), sev.isna().sum())
```

> **Observed in the supplied files:** the frequency CSV has 678,013 policies; the severity CSV has 26,639 claim rows. The inspection found 195 severity rows across six policy IDs absent from frequency, and a matched-policy claim-count discrepancy. These are reconciliation flags, not automatic deletion rules. Preserve the audit output and explain the final inclusion/exclusion policy.

After reviewing the audit and source documentation, create `sev_clean` explicitly as the claim rows accepted by your written reconciliation rule. The correct inclusion rule cannot be inferred from a row-count mismatch alone. If you cannot justify a rule, report the limitation and avoid presenting a joint-model result as fully reconciled.

Also check duplicate rows, duplicate IDs, impossible categories, negative/zero amounts, unusual exposure, and out-of-range values. Investigate extreme values before deciding whether they are errors or genuine high-cost risks.

4. Exploratory data analysis

Use descriptive statistics, distribution plots, missingness tables, and portfolio summaries. For insurance data, report exposure and claims together: a raw claim count by group can be misleading when groups have different exposure.

```
import matplotlib.pyplot as plt
import seaborn as sns

print(freq.describe(include="all").T)
print("Zero-claim policy share:", freq["ClaimNb"].eq(0).mean())
print("Total exposure:", freq["Exposure"].sum())
print("Overall claims per policy-year:",
      freq["ClaimNb"].sum() / freq["Exposure"].sum())

by_region = freq.groupby("Region", observed=True).agg(
    policies=("IDpol", "size"),
    exposure=("Exposure", "sum"),
    claims=("ClaimNb", "sum"),
)
by_region["claim_rate"] = by_region["claims"] / by_region["exposure"]
print(by_region.sort_values("claim_rate", ascending=False))

sns.histplot(freq["ClaimNb"], discrete=True)
plt.yscale("log")
plt.title("Claim count per policy (log count scale)")
plt.tight_layout()
plt.show()
```

Explore claim counts and exposure, severity (often with a log x-axis), numerical feature distributions, category levels, and group claim rates. Plot rates using `sum(ClaimNb) / sum(Exposure)`. Avoid interpreting small-group rates without displaying their exposure or uncertainty.

The existing `Data_Analysis/generate_eda_report.py` and `Reports/EDA_Report.html` can be used as a starting point. Verify its reconciliation counts against the audit above before relying on them.

5. Split data and prevent leakage

Make a reproducible train/validation/test split before fitting encoders, imputers, scalers, feature selection, or target-derived transformations. Since multiple claim rows belong to one policy, split at policy ID level and then assign all related severity rows to the same partition. The dataset has no policy date specified here, so a temporal split is not available unless additional time data is found.

```
from sklearn.model_selection import train_test_split

train_ids, test_ids = train_test_split(
    freq["IDpol"], test_size=0.20, random_state=SEED
)
train_ids, valid_ids = train_test_split(
    train_ids, test_size=0.20, random_state=SEED
)

train = freq[freq["IDpol"].isin(train_ids)].copy()
valid = freq[freq["IDpol"].isin(valid_ids)].copy()
test = freq[freq["IDpol"].isin(test_ids)].copy()

# sev_clean must contain only claim records accepted under the documented
# reconciliation protocol. Apply the same policy-ID partitions to it.
sev_train = sev_clean[sev_clean["IDpol"].isin(train_ids)].copy()
```

Use validation data for model/parameter selection. Keep the test set untouched until the modeling decisions are fixed. If a target is derived from claim rows, aggregate and join only after the policy split to avoid leakage across partitions.

6. Actuarial baselines: Poisson frequency and Gamma severity

Poisson GLM with exposure offset

The offset makes expected claim count proportional to exposure while modeling the annual claim rate with a log link. This is the primary frequency baseline.

```
import statsmodels.api as sm
import statsmodels.formula.api as smf
import numpy as np

formula = (
    "ClaimNb ~ VehPower + VehAge + DrivAge + BonusMalus + Density "
    "+ C(VehBrand) + C(VehGas) + C(Area) + C(Region)"
)
poisson = smf.glm(
    formula=formula,
    data=train,
    family=sm.families.Poisson(),
    offset=np.log(train["Exposure"]),
).fit()

valid_expected_claims = poisson.predict(
    valid, offset=np.log(valid["Exposure"])
)
valid_frequency_rate = valid_expected_claims / valid["Exposure"]
print(poisson.summary())
```

Check convergence, coefficient uncertainty, residual/deviance diagnostics, and whether the Poisson variance assumption is reasonable. A GLM is a benchmark, not an unquestioned ground truth.

Gamma severity model

Fit severity only to validated positive claim records. Reuse policy features but ensure each observed claim remains a claim-level row. Check right skew and large losses; compare plausible severity distributions rather than relying on one fit.

```
sev_train = sev_train.merge(
    train.drop(columns=["ClaimNb", "Exposure"]),
    on="IDpol", how="inner", validate="many_to_one"
)
severity_formula = (
    "ClaimAmount ~ VehPower + VehAge + DrivAge + BonusMalus + Density "
    "+ C(VehBrand) + C(VehGas) + C(Area) + C(Region)"
)
gamma = smf.glm(
    formula=severity_formula,
    data=sev_train,
    family=sm.families.Gamma(
        link=sm.families.links.Log()
    ),
).fit()
```

Do not impute zero severity for policies without claims. A policy with no claim has zero aggregate cost, but its severity is unobserved, not zero.

7. Machine learning and pure-premium modeling

Exposure-weighted Poisson boosting

A practical gradient-boosting setup predicts claim rate per exposure unit and weights each row by its exposure. For a Poisson loss, this is equivalent (up to terms independent of model parameters) to modeling claim counts with exposure as an offset. Compare the weighted implementation against the GLM benchmark.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from xgboost import XGBRegressor

numeric = ["VehPower", "VehAge", "DrivAge", "BonusMalus", "Density"]
categorical = ["VehBrand", "VehGas", "Area", "Region"]
features = numeric + categorical
preprocess = ColumnTransformer([
    ("num", "passthrough", numeric),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical),
])
boost = Pipeline([
    ("prep", preprocess),
    ("model", XGBRegressor(
        objective="count:poisson", n_estimators=500, max_depth=4,
        learning_rate=0.04, subsample=0.8, colsample_bytree=0.8,
        reg_lambda=1.0, random_state=SEED, n_jobs=-1,
    )),
])
train_rate = train["ClaimNb"] / train["Exposure"]
boost.fit(
    train[features], train_rate,
    model__sample_weight=train["Exposure"],
)
valid_rate = boost.predict(valid[features])
valid_rate = np.maximum(valid_rate, 1e-10)
valid_expected_claims = valid_rate * valid["Exposure"]
```

Keep preprocessing inside the pipeline so it is learned only on training data. Tune depth, learning rate, number of trees, and regularization on validation data. Check your installed XGBoost version and its parameter behavior. Compare LightGBM only if available and justified; do not select a model by training score.

Combined pure premium and direct Tweedie alternative

For a frequency-severity model, multiply predicted annual frequency by predicted average severity. For a direct Tweedie model, first create a reconciled policy-level aggregate loss target, then model loss per exposure with exposure weights.

```
# sev_clean is the claim table accepted under the documented protocol.
policy_loss = sev_clean.groupby("IDpol")["ClaimAmount"].sum()
policy = freq.join(policy_loss.rename("TotalLoss"), on="IDpol")
policy["TotalLoss"] = policy["TotalLoss"].fillna(0)
policy["PurePremium"] = policy["TotalLoss"] / policy["Exposure"]

from sklearn.linear_model import TweedieRegressor
tweedie = Pipeline([
    ("prep", preprocess),
    ("model", TweedieRegressor(
        power=1.5, link="log", alpha=0.1, max_iter=1000
    )),
])
tweedie.fit(
    train[features],
    train["PurePremium"],
    model__sample_weight=train["Exposure"],
)

# Frequency-severity prediction, with severity model already fitted:
annual_rate = boost.predict(test[features])
mean_claim_cost = severity_model.predict(test[features])
combined_pure_premium = annual_rate * mean_claim_cost
```

The illustrative code assumes `severity_model` is a fitted model whose prediction is mean claim amount. Build policy loss only from the reconciled claim records; unresolved claims make the target unreliable. Tune Tweedie power within the compound Poisson-Gamma range (between 1 and 2) using training/validation data.

Neural networks are optional, not mandatory. Add one only after you have a reliable baseline, a policy-level split, appropriate exposure handling, and enough validation capacity to tune it fairly.

8. Evaluation, calibration, and model selection

Evaluate on the correct target scale. For frequency, compare observed counts with expected counts or compare rates with exposure weights. For severity, assess positive claim rows. For pure premium, use reconciled policy total loss and exposure. Report more than one metric and explain what each measures.

```
from sklearn.metrics import mean_poisson_deviance, mean_tweedie_deviance

frequency_deviance = mean_poisson_deviance(
    valid["ClaimNb"] / valid["Exposure"],
    valid_rate,
    sample_weight=valid["Exposure"],
)
pure_premium_deviance = mean_tweedie_deviance(
    valid["PurePremium"],
    valid_premium_prediction,
    power=1.5,
    sample_weight=valid["Exposure"],
)
print("Exposure-weighted Poisson deviance:", frequency_deviance)
print("Exposure-weighted Tweedie deviance:", pure_premium_deviance)
```

Also report severity deviance/MAE on claim records, aggregate predicted versus observed losses, and results by validation set. Avoid using percentage error measures that become undefined or misleading at zero observed cost.

Calibration and lift by risk decile

```
cal = valid[["Exposure", "ClaimNb"]].copy()
cal["pred_rate"] = valid_rate
cal["predicted_claims"] = cal["pred_rate"] * cal["Exposure"]
cal["decile"] = pd.qcut(
    cal["pred_rate"].rank(method="first"), 10, labels=False
)
summary = cal.groupby("decile", observed=True).agg(
    exposure=("Exposure", "sum"),
    observed_claims=("ClaimNb", "sum"),
    predicted_claims=("predicted_claims", "sum"),
)
summary["observed_rate"] = summary["observed_claims"] / summary["exposure"]
summary["predicted_rate"] = summary["predicted_claims"] / summary["exposure"]
print(summary)
```

Plot observed and predicted rates or costs across deciles, display exposure/claim counts, and state how the Gini index is calculated. For the final result, calculate metrics once on the held-out test set after model selection. Use bootstrap intervals or repeated validation where feasible to communicate uncertainty.

9. Interpretability and group analysis

Use SHAP for a selected tree model to examine global importance, direction, and possible interactions. Explain that feature attribution describes model behavior; it does not establish causality. For one-hot encoded features, report readable original feature groupings as well as encoded-level effects.

```
import shap

sample = valid[features].sample(
    min(500, len(valid)), random_state=SEED
)
encoded = boost.named_steps["prep"].transform(sample)
if hasattr(encoded, "toarray"):
    encoded = encoded.toarray()
tree = boost.named_steps["model"]
explainer = shap.TreeExplainer(tree)
values = explainer.shap_values(encoded)
names = boost.named_steps["prep"].get_feature_names_out()
shap.summary_plot(values, encoded, feature_names=names)
```

Check SHAP compatibility with your library versions and model objective. State the sample and output scale. Analyze calibration and errors across age bands and regions with group sample sizes and exposure. Explain metric choices and limitations; differences are signals for investigation, not proof of unfairness or causation.

10. Final fit, save, score, and report

Once the model and parameters are selected, refit the chosen pipeline using training plus validation data, while retaining the untouched test results as the reported holdout estimate. Save the full preprocessing/model pipeline, metadata, versions, feature definition, and reconciliation decisions together.

```
import joblib
from datetime import datetime, timezone

bundle = {
    "frequency_model": final_frequency_model,
    "severity_model": final_severity_model,
    "features": features,
    "target_definition": "claim rate per exposure unit",
    "seed": SEED,
    "created_utc": datetime.now(timezone.utc).isoformat(),
```

```
    "pandas_version": pd.__version__,
    "sklearn_version": sklearn.__version__,
    "reconciliation_note": "Insert the documented final rule here",
}
joblib.dump(bundle, REPORT_DIR / "motor_pure_premium_model.joblib")
```

For new policies, validate required columns, category handling, positive exposure, and plausible input ranges. A technical pure premium is expected claim cost only; commercial premiums may require loadings, taxes, minimums, and regulatory constraints outside this model.

```
def score_policies(bundle, policy_features, exposure):
    missing = set(bundle["features"]) - set(policy_features.columns)
    if missing:
        raise ValueError(f"Missing required features: {sorted(missing)}")
    exposure = np.asarray(exposure, dtype=float)
    if (exposure.ndim != 1 or len(exposure) != len(policy_features)
            or not np.isfinite(exposure).all() or (exposure <= 0).any()):
        raise ValueError("Exposure must be a positive finite value per policy")

    rate = bundle["frequency_model"].predict(
        policy_features[bundle["features"]]
    )
    severity = bundle["severity_model"].predict(
        policy_features[bundle["features"]]
    )
    pure_premium = rate * severity
    return pd.DataFrame({
        "frequency_rate": rate,
        "mean_claim_severity": severity,
        "pure_premium_per_exposure": pure_premium,
        "expected_loss_for_exposure": pure_premium * exposure,
    })
```

Adapt input validation to the actual model interface. Raise clear errors for invalid data; do not silently substitute default predictions.

11. Practical checklist and research cautions

- **Data:** preserve raw files; validate schema, IDs, exposure, claims, missingness, duplicates, and file linkage.
- **Reconciliation:** document every unmatched ID and claim-count mismatch, and justify the analysis population.
- **Exposure:** use an offset or mathematically equivalent exposure-weighted rate objective for frequency; evaluate on a consistent scale.
- **Split:** separate policies before feature learning; keep all claim rows for a policy in the same partition.
- **Baselines:** report Poisson frequency, a documented severity model, and preferably direct Tweedie pure premium.
- **Validation:** select using validation data; use the test set once for final comparison.
- **Communication:** provide deviance, calibration/lift, Gini definition, exposure by segment, and uncertainty.
- **Interpretation:** treat SHAP as model explanation, not causality; discuss fairness limitations and group sizes.
- **Reproduction:** fix seeds, save settings/versions, separate code from data, and cite the CASdatasets source (Dutang and Charpentier) in the paper.

> **Suggested implementation order:** (1) audit and reconcile, (2) EDA, (3) policy-level splits, (4) GLM baselines, (5) boosted models and Tweedie, (6) validation/calibration, (7) SHAP and group analysis, (8) final test/report and reproducible scoring bundle.

This guide is a project-specific coding roadmap. Confirm package versions, metric conventions, data provenance, and actuarial assumptions before reporting research conclusions.